📘 GUIA RÁPIDO NEBULA LANGUAGE

Como Começar e Usar a Linguagem .neb

🚀 PARTE 1: PRIMEIROS PASSOS (Onde Começar)

1.1 Estrutura do Projeto

```
TRABALHO/
├── docs/                          # Documentação
│   ├── ESPECIFICACAO_NEBULA.md    # Especificação técnica completa
│   ├── INSTRUCOES_EXECUCAO.md     # Instruções detalhadas
│   └── README.md                  # Documentação geral
├── examples/                      # Exemplos de código
│   ├── tests/                     # Arquivos de teste
│   │   ├── teste_precedencia.neb
│   │   ├── teste_condicional.neb
│   │   ├── teste_erros_recuperaveis.neb
│   │   └── teste_invalido.neb
│   └── programa_exemplo.neb       # Programa completo
└── src/                           # Código fonte
    ├── nebula.py                  # Interpretador principal
    └── run_tests.py               # Script de testes automáticos
```

1.2 Requisitos Mínimos

- Python 3.7+ instalado
- Nenhuma biblioteca externa necessária
- Arquivos salvos com codificação UTF-8

1.3 Comando Único para Testar Abra o terminal na pasta TRABALHO e execute:

```
python src/nebula.py examples/programa_exemplo.neb
```

Pronto! Se aparecer a saída com cálculos estatísticos, está funcionando! ✅

📝 PARTE 2: COMO USAR A LINGUAGEM .neb

2.1 Olá Mundo (Seu Primeiro Programa) Crie um arquivo `meu_programa.neb`:

```
// Meu primeiro programa Nebula
exibir("Ola, Mundo!");
```

Execute:

```
python src/nebula.py meu_programa.neb
```

Saída:

```
Ola, Mundo!
```

2.2 Declaração de Variáveis Use a palavra-chave `criar` (não use let ou var):

```
// Declaração simples
criar nome;
criar idade = 25;
criar preco = 19.99;
criar ativo = verdadeiro;
criar mensagem = "Bem-vindo!";
```

Regras:

- Toda declaração termina com `;`
- Nomes podem conter letras, números e `_`
- Não pode começar com número

2.3 Operadores Aritméticos

Operador	Descrição	Exemplo
`+`	Adição	`criar soma = 5 + 3;`
`-`	Subtração	`criar diff = 10 - 4;`
`*`	Multiplicação	`criar prod = 6 * 7;`
`/`	Divisão	`criar div = 20 / 4;`
`%`	Módulo (resto)	`criar resto = 10 % 3;`
`^`	Potenciação	`criar pot = 2 ^ 3;`

Precedência (ordem de execução):

1. `()` - Parênteses primeiro
2. `^` - Potenciação (direita para esquerda)
3. `* / %` - Multiplicação/Divisão/Módulo
4. `+ -` - Adição/Subtração

```
// Exemplo de precedência
criar resultado = 2 + 3 * 4;    // Resultado: 14 (não 20!)
criar correto = (2 + 3) * 4;    // Resultado: 20
criar potencia = 2 ^ 3 ^ 2;     // Resultado: 512 (2^(3^2), não (2^3)^2)
```

2.4 Operadores Relacionais e Lógicos

Relacionais:

Operador	Significado
`==`	Igual a
`!=`	Diferente de
`<`	Menor que
`<=`	Menor ou igual
`>`	Maior que
`>=`	Maior ou igual

Lógicos (estilo C):

Operador	Significado
`&&`	E lógico
`||`	OU lógico
`!`	NÃO lógico

```
criar a = 10;
criar b = 20;

// Comparações
criar igual = (a == b);         // falso
criar maior = (a > 5);          // verdadeiro

// Lógicos
criar condicao = (a > 5) && (b < 30);   // verdadeiro
criar alternativa = !falso;             // verdadeiro
```

2.5 Estrutura Condicional (if/else) Use `se` e `senao` com blocos `inicio ... fim`:

```
criar nota = 7.5;

se (nota >= 7.0) inicio
    exibir("Aprovado");
fim
senao inicio
    exibir("Reprovado");
fim
```

Condicional aninhada:

```
criar idade = 25;

se (idade < 13) inicio
    exibir("Crianca");
fim
senao inicio
    se (idade < 20) inicio
        exibir("Adolescente");
    fim
    senao inicio
        exibir("Adulto");
    fim
fim
```

2.6 Laço de Repetição (while) Use `enquanto` com blocos:

```
criar contador = 1;

enquanto (contador <= 5) inicio
    exibir(contador);
    contador = contador + 1;
fim
```

Saída:

```
1
2
3
4
5
```

2.7 Comentários

```
// Comentário de uma linha

/*
 * Comentário de bloco
 * Pode ter várias linhas
 */
```

2.8 Saída de Dados Use `exibir()` para mostrar valores:

```
criar nome = "Maria";
criar idade = 30;

exibir("Nome:");        // Imprime: Nome:
exibir(nome);           // Imprime: Maria
exibir(idade);          // Imprime: 30

// Concatenação automática com números e strings
exibir("Idade: " + idade);  // Imprime: Idade: 30
```

🧪 PARTE 3: TESTES PRONTOS

Execute todos os testes para verificar se está funcionando:

Teste 1: Precedência de Operadores

```
python src/nebula.py examples/tests/teste_precedencia.neb
```

Verifica: Ordem correta das operações matemáticas

Teste 2: Condicionais

```
python src/nebula.py examples/tests/teste_condicional.neb
```

Verifica: Funcionamento de se/senao aninhados

Teste 3: Recuperação de Erros

```
python src/nebula.py examples/tests/teste_erros_recuperaveis.neb
```

Verifica: O interpretador continua após erros sintáticos

Teste 4: Código Inválido

```
python src/nebula.py examples/tests/teste_invalido.neb
```

Verifica: Detecção de erros léxicos e sintáticos

Teste 5: Programa Completo

```
python src/nebula.py examples/programa_exemplo.neb
```

Verifica: Cálculo estatístico completo (média, variância, etc.)

🛠️ PARTE 4: DICAS E RESOLUÇÃO DE PROBLEMAS

Modo Verbose (Detalhado) Para ver tokens e AST gerada:

```
python src/nebula.py examples/programa_exemplo.neb --verbose
```

Erros Comuns

Erro	Causa	Solução
Esperado `;`	Faltou ponto e vírgula	Adicione `;` no final da linha
Esperado `inicio`	Bloco sem delimitador	Use `inicio ... fim`
Variável não declarada	Usou variável antes de criar	Declare com `criar nome;` primeiro
Caractere inválido	Símbolo não reconhecido	Verifique operadores (use `&&` não `and`)

Executar Todos os Testes de Uma Vez

```
python src/run_tests.py
```

📋 RESUMO DA SINTAXE NEBULA

Elemento	Sintaxe	Exemplo
Declaração	`criar nome [= valor];`	`criar x = 10;`
Atribuição	`nome = valor;`	`x = 20;`
Condicional	`se (cond) inicio ... fim [senao inicio ... fim]`	`se (x > 5) inicio exibir("OK"); fim`
Repetição	`enquanto (cond) inicio ... fim`	`enquanto (x < 10) inicio ... fim`
Saída	`exibir(expr);`	`exibir("Resultado: " + x);`
Bloco	`inicio ... fim`	`inicio criar a = 1; exibir(a); fim`
Comentário	`//` ou `/* */`	`// isto é um comentário`

🎯 CHECKLIST PARA COMEÇAR

- [ ] Python 3.7+ instalado
- [ ] Arquivos do projeto na pasta TRABALHO
- [ ] Executou: `python src/nebula.py examples/programa_exemplo.neb`
- [ ] Viu a saída com “Media:”, “Variancia:”, etc.
- [ ] Criou seu próprio arquivo `.neb`
- [ ] Testou com `exibir("Funcionou!");`

Pronto! Agora você sabe usar a linguagem Nebula! 🌌

📞 COMANDOS DE REFERÊNCIA RÁPIDA

```
# Executar programa exemplo
python src/nebula.py examples/programa_exemplo.neb

# Executar com detalhes
python src/nebula.py examples/programa_exemplo.neb --verbose

# Executar todos os testes
python src/run_tests.py

# Ajuda
python src/nebula.py --help
```

Documento criado para a disciplina N477 - Construção de Interpretadores e Compiladores
Unitri - 2026